

分布式缓存

缓存雪崩（Cache Avalanche）

概念：缓存雪崩通常发生在缓存中大量数据同时失效或缓存系统因故障整体宕机时。由于大量请求直接转向数据库或其他后端存储，导致数据库压力剧增，性能急剧下降，甚至崩溃。就像一场突如其来的雪崩，你的应用开始变得奇慢无比，甚至完全停止响应。

发生情况：

1、同步过期：如果你将大量缓存设置为在同一时间过期，这就像定时炸弹一到时间就会爆炸。突然间，所有数据都需要重新加载到缓存中，这时候所有的请求都会转到数据库上，导致瞬间流量激增。

2、系统重启：有时系统维护或意外的服务重启会导致所有缓存失效。当服务再次上线，所有的请求都会涌向空无一物的缓存，然后转向数据库，形成了一场人造的“雪崩”。

3、Redis 服务宕机：虽然 Redis 非常稳定，但没有什么是是不可能的。硬件故障、网络问题或配置错误都可能导致 Redis 服务不可用。当这个守护着性能的壁垒倒下，雪崩就会随之而来。

4、热点 key 消失：在某些情况下，特定的热点 key（被大量频繁访问的 key）如果失效或被删除，也会导致相应的大量请求直接落到数据库上，造成局部的雪崩效应。

表现：

1、响应延迟增加：

场景描述：想象一下，用户发送请求期望迅速得到响应，但是因为缓存雪崩，这些请求都不得不等待数据库缓慢地处理。

影响：用户体验大打折扣，原本几毫秒内可以得到的结果，现在可能需要数秒甚至更久。

2、系统负载激增：

场景描述：数据库原本靠缓存作为缓冲，突然间所有请求都直接涌向数据库，这就像一条宁静的河流突然变成了狂暴的洪水。

影响：系统资源消耗激增，处理能力迅速饱和，导致整个应用的性能下降。

3、服务完全不可用：

场景描述：在极端情况下，数据库可能因为压力过大而完全崩溃，就像被雪崩掩埋的小镇，一切都停止了运作。

影响：应用或服务完全不可用，用户无法完成任何操作，业务运行停滞。

预防或补救方法：

设置合理的 TTL(Time To Live)，避免大量缓存数据同时失效。

使用缓存集群或分布式缓存，避免单点故障。

实现缓存预热机制，提前加载缓存数据。

使用队列或消息队列来处理缓存失效后的请求。

缓存击穿（Cache Penetration）

缓存击穿通常发生在高并发系统中，当某个被频繁访问的数据（热点 key）在缓存中的有效期过了，恰好在这个时候有大量的并发请求需要访问这个数据。由于缓存中的数据已经失效，这些并发请求就会直接转发到数据库上，如果数据库的处理能力不足以应对这种突然增加的压力，就可能导致系统响应缓慢甚至崩溃。

发生情况：

- 1、某个热点数据的缓存失效或过期。
- 2、应用程序或数据库进行大规模更新，导致热点数据的缓存失效。

表现：

- 1、系统性能下降，响应时间增加。
- 2、数据库或后端系统的负载增加，可能导致崩溃。
- 3、用户可能会看到错误或 `timeout`。
- 4、系统性能急剧下降，由于大量请求直接打到数据库上，数据库负载增加，可能导致系统整体性能下降，包括 CPU 使用率升高、内存占用增加等。

预防或补救方法：

1. 对于热点数据设置永不过期或较长的过期时间。
2. 使用互斥锁（如 Redis 的 `SETNX` 命令），确保同一时间只有一个请求去数据库获取数据，其他请求等待或获取缓存中的旧数据。

3. 使用布隆过滤器预先判断数据是否存在，不存在的数据直接返回，不继续访问数据库。

4. 当发生缓存击穿时，可以通过限流、降级等策略保护数据库，确保系统稳定。

总之，缓存雪崩和击穿都是由于缓存数据的失效或过期引起的系统性能问题。通过设置合理的 TTL、使用缓存集群或分布式缓存、实现缓存预热机制、使用缓存锁机制等方法，可以预防或补救这些问题。

要在三台 Linux 服务器上实现一个分布式缓存系统，可以采用以下步骤和技术实现。这种实现方式考虑了数据一致性、可扩展性和高可用性。

如何实现分布式缓存

步骤一：环境准备

1. 准备三台 Linux 服务器：假设三台服务器的 IP 地址分别为 192.168.1.1、192.168.1.2 和 192.168.1.3。

2. 安装必要的软件：确保所有服务器上都安装了相应的依赖和软件，如 Redis 或 Memcached，以及配置好的网络连接。

步骤二：选择分布式缓存方案

两种常见的分布式缓存实现方式：

1. Redis 集群

2. Memcached 分布式缓存

这里以 Redis 集群为例

步骤三：安装和配置 Redis

1. 下载并安装 Redis:

在每台服务器上执行以下命令:

```
wget http://download.redis.io/redis-stable.tar.gz
tar xvzf redis-stable.tar.gz
cd redis-stable
make
```

2. 配置 Redis:

在每台服务器上创建并编辑配置文件 `redis.conf`, 主要配置如下:

```
port 7000
cluster-enabled yes
cluster-config-file nodes-7000.conf
cluster-node-timeout 5000
appendonly yes
```

3. 启动 Redis 实例:

在每台服务器上启动 Redis 实例:

```
redis-server redis.conf
```

步骤四：创建 Redis 集群

1. 使用 redis-cli 创建集群：

在任意一台服务器上，使用 `redis-cli` 创建集群。假设在 192.168.1.1 上：

```
redis-cli --cluster create 192.168.1.1:7000 192.168.1.2:7000 192.168.1.3:7000 --cluste
```

```
redis-cli --cluster create 192.168.1.1:7000 192.168.1.2:7000  
192.168.1.3:7000 --cluster-replicas 1
```

这里 `--cluster-replicas 1` 指定每个主节点有一个副本。

2. 验证集群状态：

在任意一台服务器上使用 `redis-cli` 查看集群状态：

```
redis-cli -c -p 7000 cluster info
```

步骤五：使用和管理 Redis 集群

1. 连接集群：

应用程序连接 Redis 集群时，需要支持 Redis 集群协议的客户端库。例如，Python 使用 `redis-py-cluster`，Java 使用 `JedisCluster`。

2. 数据读写：

分布式缓存系统会自动根据 `key` 的 `hash` 值将数据分布到不同的节点上，并处理节点间的数据复制和故障转移。

为什么这样实现

1. 数据一致性：

Redis 集群通过分片机制将数据分布到多个节点上，并支持副本，确保数据的高可用性和一致性。

2. 高可用性：

Redis 集群支持主从复制和自动故障转移，当某个节点发生故障时，集群会自动将请求转发到其他可用节点。

3. 扩展性：

Redis 集群支持水平扩展，可以根据需要增加或减少节点，方便处理增长的缓存需求。

4. 性能优化：

分布式缓存可以将缓存压力分散到多个节点上，提高缓存的读写性能，并减少单个节点的负载。

通过以上步骤，利用 Redis 集群可以在三台 Linux 服务器上实现一个高效的分布式缓存系统，保证数据的高可用性、一致性和系统的扩展性。